

Praxis: Race Conditions erkennen und beheben

Fehler provozieren, Ursache beweisen,
Fix verifizieren

2.09

Reproduzierbare Fehler als Basis

- | | | |
|----|--------------|----------------------------------|
| 01 | PROVOZIEREN | Parallelität und Last erhöhen |
| 02 | BEOBACHTEN | Symptom und Kontext erfassen |
| 03 | BEWEISEN | KCSAN, Lockdep und Codepfad |
| 04 | BEHEBEN | Passendes Primitiv einsetzen |
| 05 | VERIFIZIEREN | Test unter denselben Bedingungen |

Nur ein reproduzierbarer Test zeigt später, ob der Fix wirkt.

Check-then-act verletzt die Invariante

FEHLERHAFTER PFAD

```
static unsigned int free_slots = 1;

int reserve_slot(void)
{
    if (free_slots == 0)
        return -ENOSPC;
    free_slots--;
    return 0;
}
```

01 Invariante

$0 \leq \text{free_slots} \leq \text{MAX_SLOTS}$

02 Fehlerbild

Zwei erfolgreiche Reservierungen für denselben Slot.

03 Ursache

Prüfen und Ändern sind nicht gemeinsam atomar.

Die Invariante braucht eine gemeinsame Synchronisationsgrenze.

Parallelität macht den seltenen Fehler sichtbar

STRESSTEST

```
# Zwei konkurrierende Worker
for i in $(seq 1 1000); do
    echo reserve > /dev/race_demo &
    echo reserve > /dev/race_demo &
    wait
done

dmesg | tail -n 50
```

LAST ERHÖHEN

- Mehr CPUs
- Viele Wiederholungen
- Gleichzeitiger Start

KONSTANT HALTEN

- Gleicher Kernel
- Gleiche Konfiguration
- Gleicher Test

Reproduktion und Fix-Test verwenden dieselben Bedingungen.

Kontext macht Symptome diagnostizierbar

INSTRUMENTIERUNG

```
pr_warn_ratelimited(
    "reserve: cpu=%d pid=%d before=%u after=%u\n",
    raw_smp_processor_id(), current->pid,
    before, free_slots);

WARN_ON_ONCE(free_slots > MAX_SLOTS);
```

ERWARTUNG

Maximal eine erfolgreiche Reservierung

BEOBACHTUNG

Zähler oder Erfolgsmeldungen widersprechen

KONTEXT

CPU, Task, IRQ-Zustand, Stacktrace

Symptom, Ausführungskontext und Stacktrace gemeinsam sichern.

KCSAN belegt Data Race

KERNEL-KONFIGURATION

```
CONFIG_KCSAN=y  
CONFIG_KCSAN_STRICT=y  
CONFIG_DEBUG_INFO=y  
  
make -j$(nproc)  
boot + Stresstest
```

KCSAN-REPORT

```
BUG: KCSAN: data-race  
write to free_slots by task 142  
    reserve_slot+0x..  
read from free_slots by task 143  
    reserve_slot+0x..
```

REPORT SICHERN

Adresse, Zugriffstyp, beide Stacktraces und Testkommando

Ein fehlender Report ist kein Beweis für Fehlerfreiheit.

KCSAN und Lockdep ergänzen sich

KCSAN	Datenrennen	Konkurrierende Reads/Writes
LOCKDEP	Lock-Regeln	Reihenfolge, Kontext, API
ASSERTIONS	Invarianten	Ungültige Zustände
STRESSTEST	Reproduktion	Timing und Last
REVIEW	Semantik	Lebensdauer und Vertrag

Ein fehlender Report ist kein Beweis für Fehlerfreiheit.

Vor dem Fix steht der Synchronisationsvertrag

ZUSTAND

Welche Felder gehören zur Invariante?

ZUGRIFFSPFADE

Wer liest, ändert oder gibt frei?

KONTEXT

Prozess, Workqueue, Softirq oder Hardirq?

LEBENSDAUER

Wann kann das Objekt verschwinden?

Vertrag: `free_slots` wird nur unter `slots_lock` geprüft und verändert.

Ein Mutex macht Prüfen und Ändern unteilbar

MUTEX-FIX

```
static DEFINE_MUTEX(slots_lock);

int reserve_slot(void)
{
    int ret = 0;
    mutex_lock(&slots_lock);
    if (!free_slots)
        ret = -ENOSPC;
    else
        free_slots--;
    mutex_unlock(&slots_lock);
    return ret;
}
```

01 Prüfen

Alle beteiligten Felder liegen innerhalb derselben Sektion.

02 Kontext

Nur verwenden, wenn jeder Aufrufer schlafen darf.

IRQ-Sharing verlangt irqsave-Symmetrie

IRQ-SICHERER FIX

```
unsigned long flags;  
spin_lock_irqsave(&slots_lock, flags);  
if (!free_slots)  
    ret = -ENOSPC;  
else  
    free_slots--;  
spin_unlock_irqrestore(&slots_lock, flags);
```

KURZ HALTEN

- Kein Schlafen
- Kein I/O
- Keine langen Schleifen

PAAR BILDEN

- irqsave mit irqrestore
- bh mit bh
- lock mit unlock

Lock und Unlock müssen denselben Interruptschutz verwenden.

Atomic schützt einzelne Zustandswechsel

ZUSTANDSÜBERGANG

```
atomic_t state = ATOMIC_INIT(FREE);  
  
if (atomic_cmpxchg(&state, FREE, BUSY) != FREE)  
    return -EBUSY;  
  
return 0;
```

GEEIGNET

- Ein Wert
- Ein Übergang
- Klare Zustandsmaschine

NICHT GEEIGNET

- Mehrere Felder
- Check-then-act über Objekte
- Komplexe Lebensdauer

Atomic passt nur zu einem klar abgegrenzten Einzelzustand.

Wait Queues brauchen geschützte Bedingungen

WARTEN

```
wait_event_interruptible(  
    slots_wq,  
    READ_ONCE(free_slots) > 0);  
ret = reserve_slot();
```

FREIGEBEN UND WECKEN

```
mutex_lock(&slots_lock);  
free_slots++;  
mutex_unlock(&slots_lock);  
  
wake_up_interruptible(&slots_wq);
```

Zustand ändern Lock lösen Wartende wecken

Die Bedingung entscheidet. Das Wake-up fordert nur eine erneute Prüfung an.

Race Conditions gefährden die Lebensdauer

01 Referenz erwerben

get / refcount_inc_not_zero

02 Objekt verwenden

Pointer bleibt gültig

03 Referenz lösen

put / refcount_dec_and_test

04 Freigeben

Erst nach letzter Referenz

Pointer-Sichtbarkeit und Objektlebensdauer sind zwei verschiedene Verträge.

Der Fix muss den Stresstest bestehen

BUILD	Warnings und passende Debug-Optionen
FUNKTION	Erfolg, Fehler und Randwerte
PARALLELITÄT	Gleicher Stresstest wie vor dem Fix
KCSAN	Kein Report im relevanten Zugriffspfad
LOCKDEP	Keine Kontext- oder Reihenfolgenwarnung
REVIEW	Vertrag im Code dokumentiert

Erfolgskriterium: Die Invariante hält unter Last ohne neue Warnungen.

Race Conditions systematisch beheben

REPRODUZIEREN	Parallelität kontrolliert erhöhen
BELEGEN	Logs, Stacktraces und Sanitizer sichern
INVARIANTE	Gemeinsam konsistente Werte benennen
KONTEXT	Schlafbarkeit, IRQs und Lebensdauer klären
BEHEBEN	Mutex, Spinlock, Atomic, Refcount oder RCU wählen
VERIFIZIEREN	Funktion, Stress, KCSAN und Lockdep wiederholen

Der Test, der den Fehler zeigt, wird zum Regressionstest.